

Module 7: Lesson 2

Momentum Timing with Linear Regression as a NN



Outline

- ▶ Smart beta / factor investing
- ▶ Linear regression as a neural network for factor timing
- ▶ Backtesting the 'Mom timing' strategy

Factor investing

In this lesson, which is focused on working with the accompanying Jupyter Notebook, we will implement a linear regression model using neural networks in the Tensorflow-Keras framework.

- ▶ **What are we going to use our neural network for?**

We will build a predictive model to develop a trading strategy based on factor investing (or smart beta).

- ▶ **Factor investing?**

- **What is a factor?**

Essentially, a characteristic that systematically explains the cross-section of stock returns (e.g., size, value, momentum, etc.).

- **What is factor investing?**

These factors help explain the cross-section of stock returns, so we can invest following them.

Value example: a portfolio that goes long on 10% firms with the lowest book-to-market (B/M) and short on the 10% with the highest B/M delivers an above-market return (factor premium).

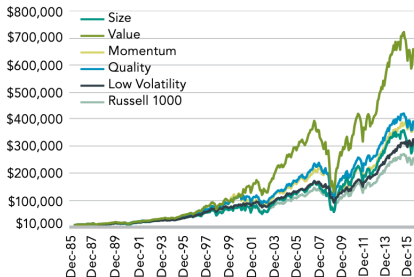
Factor investing uses a systematic approach to try and capture all these factor premiums

Factor Investing

But all these factor premiums are time-varying and a specific factor does not always outperform.

EXHIBIT 7: Growth of \$10,000 Investing in Factor Portfolios vs. the Broader Market (1985–2015)

These five factors have outperformed over time

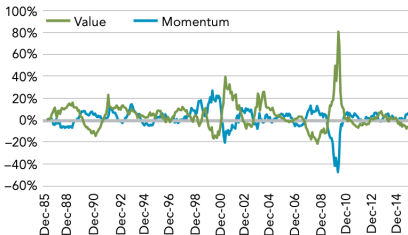


Returns are cumulative and assume reinvestment of dividends. Past performance is no guarantee of future results. Factor definitions consistent with those shown in previous exhibits. Source: FactSet, as of Mar. 31, 2016.

EXHIBIT 8: Value and Momentum Cyclicalities

Most factors are not highly correlated, so diversifying among them may improve risk-adjusted returns over time

Rolling Annual Excess Return



Value represented by the equal-weighted top quintile (by book-to-price ratio) of the Russell 1000 Index. Momentum represented by the equal-weighted top quintile (by trailing 12-month returns) of the Russell 1000 Index. Source: FactSet, as of Mar. 31, 2016.

Our factor investing strategy

All these different factors offer a source of premium that is time-varying. In this lesson's notebook, we will implement a factor-timing strategy for the **momentum** factor.

- ▶ What is **momentum**?

Refers to the documented fact that stocks that perform the best (worst) over a three- to 12-month period tend to continue to perform well (poorly) over the subsequent three to 12 months.

- ▶ What does **momentum timing** require?

Simply, that we can predict the returns that the momentum factor is going to offer in the future.

In our context, successfully timing momentum requires that our NN model is able to predict momentum returns relatively well using past momentum returns as inputs.

- ▶ What **process** are we going to follow?

1. Predictive model → Linear regression neural network
2. Develop trading strategy based on momentum timing
3. Backtesting the momentum timing trading strategy to evaluate its performance

Neural network for momentum timing

Our first step is to construct a neural network model that aims to predict momentum returns. To do so, we will gather data on daily historical momentum factor returns. Our model will then comprise:

- ▶ **Output:** Momentum factor returns for the next 60 days.
- ▶ **Inputs:** Momentum-sorted portfolio returns for:
 - 5 different time-spans: 10, 25, 60, 120, and 240 previous days.
 - 3 different portfolios: the top 10% and bottom 10% of firms by previous momentum, and a Mom portfolio that buys the top and sells the bottom decile portfolios.

This means a total of $3 \times 5 = 15$ inputs. Thus, the model will train 16 parameters: 15 weights (w), plus the bias term (b).

- Before training the model, we select 60% of observations (careful about the chronological order!) for training and the remaining 40% for test.
- We also need to perform some scaling of the variables (inputs and outputs) following the classic MinMaxScaler in sklearn.

Neural network for momentum timing

- How do we train the model? → Using a single-layer NN!

```
import tensorflow as tf

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(1))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)
model.compile(optimizer = adam, loss= 'mean_squared_error')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=2)
```

- Which optimizer are we using? → Adam

- Adam is one of the most common deep learning optimizers based on SGD.
- Adam uses leaky averaging in the gradient descent process, taking into account both the first and second moments of the gradient.
- For more on Adam, see Section 12.10 in the book by [Zhang et al.: Dive into Deep Learning](#).

Model performance and momentum timing

Once we have trained our model in sample, it is time to assess its predictive ability and whether we can use it for trading purposes.

► Assessing model fit

- We already know that the R^2 is a classic measure in linear regression to evaluate goodness of fit in a model. The in-sample R^2 we are used to is, however, of no use in here. Remember that we aimed at constructing a model that can predict out-of-sample momentum returns to develop a trading strategy.
- Hence, we will rely on Campbell and Thompson (2008) to develop an out-of-sample R^2 that captures models' predictive power in future stock returns:

$$R_{OS}^2 = 1 - \frac{\sum_{t=1}^T (r_t - \hat{r}_t)^2}{\sum_{t=1}^T (r_t - \bar{r}_t)^2}$$

- A high R_{OS}^2 is a good indicator that our model will be able to help us in timing the momentum factor.

However, note that even a poor R_{OS}^2 can have some use in practice because even if the model fails a lot at predicting outcomes, the incorrect guesses may imply small losses, while the correct guesses may convey large wins!

Model performance and momentum timing

A common way to evaluate the viability of a trading strategy (not the model's predictive power!) is to perform a **Backtest**.

► Backtesting trading strategy

- Like traveling back in time and assessing how well/bad would we have done if we followed the strategy.
- It is extremely important that we **do not cheat** when performing backtesting. That is, among other things, we can only consider information that was known at each point in time when we make the trading decisions.
- If we deployed a trading strategy that buys (sells) the momentum portfolio when our model predicts a positive (negative) return the next day, this is how we would have done:



Summary of Lesson 1

In Lesson 2, we learned how to:

- ▶ Work with neural networks (NNs) in Tensorflow Keras
- ▶ Implement a linear regression problem as an NN
- ▶ Develop a smart beta trading strategy based on the predictions of a linear regression NN model for the momentum factor
- ▶ Perform backtesting of a trading strategy

⇒ **References for this lesson:**

Chapters 12.10 on [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: Go to this lesson's associated Jupyter Notebook. There, you will work on a linear regression model using a neural network to develop a trading strategy that aims to time the momentum factor.

In the next lesson, we will introduce a much more complex NN, multilayer perceptron (MLP).